UNIVERSIDADE VEIGA DE ALMEIDA
CURSO DE ENGENHARIA DA COMPUTAÇÃO

LETÍCIA DOS REIS PRADO – 1240207303

TRABALHO:
Transpilador DOMUS para C

Rio de Janeiro/RJ
2026

LETÍCIA DOS REIS PRADO

TRABALHO:
Transpilador DOMUS para C

Trabalho para cumprimento do componente curricular Compiladores do Curso de Engenharia da Computação da Universidade Veiga de Almeida.

Orientador: Fábio Contarini Carneiro

Rio de Janeiro/RJ
2026

RESUMO

Este trabalho apresenta o desenvolvimento de um transpilador da linguagem Domus para a linguagem C, elaborado como parte da avaliação A2 da disciplina de Compiladores. O transpilador dá continuidade ao analisador léxico construído no primeiro trabalho e acrescenta as fases de análise sintática, análise semântica e geração de código. A análise sintática foi construída com o gerador automático Bison e monta a árvore sintática do programa. A análise semântica percorre essa árvore, constrói a tabela de símbolos e aplica as regras da linguagem relativas à declaração de sensores e atuadores, à exclusividade das portas, à criação implícita das variáveis e às restrições de leitura e de escrita dos dispositivos. O gerador de código escreve um programa equivalente em linguagem C padrão, com os sensores, os atuadores e as variáveis declarados como inteiros globais. Os erros são informados com a indicação do tipo e da linha em que ocorreram. Os resultados obtidos com o programa de teste confirmam o reconhecimento correto do programa fonte, a construção adequada da tabela de símbolos e a geração de um código em C que compila sem avisos.

Palavras-chave: Transpilador. Compiladores. Análise sintática. Análise semântica. Geração de código. Linguagem Domus.

LISTA DE FIGURAS

FIGURA 1 – PROGRAMA DE TESTE PROG_TESTE2.DMS 11
FIGURA 2 – TABELA DE SÍMBOLOS GERADA PARA O PROGRAMA DE TESTE 12
FIGURA 3 – CÓDIGO EM LINGUAGEM C GERADO PELO TRANSPILADOR 12
FIGURA 4 – MENSAGENS DE ERRO PRODUZIDAS PELO TRANSPILADOR 14

LISTA DE TABELAS

TABELA 1 – ALTERAÇÕES NO ANALISADOR LÉXICO 6
TABELA 2 – CORRESPONDÊNCIA ENTRE AS CONSTRUÇÕES DO DOMUS E DO C 9

SUMÁRIO

1. INTRODUÇÃO 6
2. Alterações no analisador léxico 6
3. ITENS DE PRODUÇÃO DA GRAMÁTICA ESCOLHIDOS 7
3.1. Declaração de sensor 7
3.2. Comando de atribuição 8
3.3. Comando de condição 8
4. ESTRATÉGIA ADOTADA PARA A GERAÇÃO DO CÓDIGO EM C 9
5. COMO COMPILAR E EXECUTAR O CÓDIGO 10
5.1. Requisitos: 10
5.2. Compilação 10
5.3. Execução 10
6. RESULTADO DA EXECUÇÃO DO TRANSPILADOR 11
REFERÊNCIAS 15

1. INTRODUÇÃO

Um compilador traduz um programa escrito em uma linguagem fonte para um programa equivalente em outra linguagem, em quatro fases: análise léxica, análise sintática, análise semântica e geração de código. O primeiro trabalho da disciplina tratou apenas da análise léxica da linguagem Domus. Este trabalho dá continuidade àquele e apresenta um transpilador completo, que lê um programa em Domus e gera um programa equivalente em linguagem C padrão. O analisador léxico continua sendo gerado pelo Flex, a partir do arquivo domus.l, e o analisador sintático passou a ser gerado pelo Bison, a partir do arquivo domus.y. A organização dos módulos segue a do compilador TINY (LOUDEN, 2004).

2. ALTERAÇÕES NO ANALISADOR LÉXICO

As expressões regulares não foram alteradas: todos os símbolos exigidos pela gramática já eram reconhecidos pela versão entregue no primeiro trabalho, e a função getToken() permaneceu idêntica. As alterações necessárias estão relacionadas na Tabela 1.

Tabela 1 – Alterações no analisador léxico

Alteração	Motivo
Retirada da enumeração TokenType do global.h, substituída pela inclusão do arquivo domus.tab.h.	O Bison gera os seus próprios códigos para os tokens; o analisador léxico e o sintático precisam usar os mesmos valores.
Definição da constante FIMARQ com o valor 0.	O Bison devolve zero ao chegar ao fim do arquivo e não inclui esse valor no cabeçalho que gera.
Criação de uma função yylex() estática no domus.y, que apenas devolve getToken().	Permite que o Bison obtenha os tokens sem alterar o analisador léxico do primeiro trabalho.
Mensagem do erro léxico gravada em tokenString, e não em yytext.	yytext aponta para o buffer interno do Flex, dimensionado apenas para o lexema reconhecido.

Fonte: Autoria

O arquivo global.h também recebeu as definições da árvore sintática. Em relação à estrutura TreeNode do compilador TINY, foram acrescentados os campos categoria, tipoDisp e porta, que guardam, nos nós de declaração, se o identificador é

sensor ou atuador, se o dispositivo é analógico ou digital e a porta em que foi alocado. Essas informações são exigidas pelas regras semânticas da linguagem.

3. ITENS DE PRODUÇÃO DA GRAMÁTICA ESCOLHIDOS

Foram escolhidos três itens de produção: a declaração de sensor, o comando de atribuição e o comando de condição. Os dois últimos são comandos da linguagem e os dois primeiros realizam verificação semântica.

O valor associado a cada símbolo da gramática é do tipo TreeNode *, conforme a linha #define YYSTYPE TreeNode * copiada do arquivo tiny.y. Esse tipo não comporta o lexema de um identificador nem o valor de um número; por isso esses valores são gravados em variáveis globais por ações colocadas no meio da regra, que o Bison executa antes de ler o token seguinte.

3.1. Declaração de sensor

A produção reconhece a declaração de um sensor:

sensor -> IDENTIFICADOR ":" tipo_sensor_atuador "(" "porta" NUMERO ")"

Implementação: Duas ações no meio da regra gravam o nome do sensor em savedName e o número da porta em savedPorta; o tipo é gravado em savedTipo durante a redução de tipo_sensor_atuador. A ação final monta o nó de declaração com as três informações.

Verificação semântica: A função declaraDispositivo, do arquivo analyze.c, verifica se o identificador já foi declarado e se a porta já está ocupada por outro dispositivo, acusando erro nos dois casos. O dispositivo é inserido na tabela de símbolos mesmo após o erro de porta, para evitar erros encadeados nos usos seguintes.

Geração de código: A declaração produz uma variável inteira global, emitida no início do programa em C a partir da tabela de símbolos:

```
int temperatura;      /* sensor analogico - porta 1 */
```

3.2. Comando de atribuição

A produção reconhece o comando de atribuição:

comando_atribuicao -> IDENTIFICADOR "<-" expressao

Implementação: Segue o mesmo padrão da produção assign_stmt do arquivo tiny.y: uma ação no meio da regra grava o nome do identificador e a linha, e a ação final monta o nó de atribuição com a expressão como filho.

Verificação semântica: A função gravaIdentificador cria a variável na tabela de símbolos quando ela aparece pela primeira vez, já que a linguagem não possui seção de declaração de variáveis, e acusa erro quando o identificador é um sensor, que é de somente leitura. Para que o comando x <- x + 1 com x indefinido seja acusado, a função buildSymtab analisa a expressão do lado direito antes do identificador do lado esquerdo, invertendo a ordem da varredura em pré-ordem apenas nesse caso.

Geração de código: A tradução é direta. O comando asd <- asd + 2 do programa de teste produz:

asd = (asd + 2);

3.3. Comando de condição

A produção possui duas alternativas, com e sem a parte senao:

comando_se -> "se" expressao_logica "entao" bloco "fim_se"
| "se" expressao_logica "entao" bloco "senao" bloco "fim_se"

Implementação: As duas alternativas constroem um nó do tipo IfK. O primeiro filho recebe a expressão de teste, o segundo recebe o bloco da parte entao e o terceiro, quando existe, recebe o bloco da parte senao. A ausência da parte senao é identificada pelo terceiro filho nulo.

Verificação semântica: Este item não realiza verificação própria. As verificações aplicáveis recaem sobre os identificadores da expressão de teste e sobre os comandos dos blocos, tratados pelos itens correspondentes.

Geração de código: O comando corresponde ao if da linguagem C, com os blocos delimitados por chaves em lugar da palavra fim_se. O comando do programa de teste que verifica a temperatura produz:

```
if (temperatura < 30) {
    cortina = 1;
    ventilador = 80;
} else {
    cortina = 0;
    ventilador = 0;
}
```

4. ESTRATÉGIA ADOTADA PARA A GERAÇÃO DO CÓDIGO EM C

A geração do código foi dividida em dois módulos, como no compilador TINY: o arquivo code.c reúne os utilitários de emissão, que escrevem no arquivo de saída, e o arquivo cgen.c reúne a lógica da tradução, com as funções genStmt, genExp, cGen e codeGen.

No TINY a linguagem destino é o código de máquina do simulador TM, e por isso o code.c oferece as funções emitSkip, emitBackup e emitRestore, necessárias para preencher retroativamente os endereços de desvio dos comandos de condição e de repetição. Como a linguagem destino deste trabalho é o C, que já possui as estruturas if e while, nenhum desvio precisa ser calculado e esse mecanismo foi dispensado. Em seu lugar, o code.c oferece funções de emissão de texto e o controle do nível de indentação.

O programa gerado segue a estrutura exigida pelo documento de regras: as inclusões necessárias, os sensores, os atuadores e as variáveis declarados como inteiros globais e, por último, a função main com os comandos traduzidos. As declarações globais são obtidas da tabela de símbolos, e não da árvore, porque as variáveis da linguagem Domus não possuem declaração no programa fonte. A correspondência entre as construções das duas linguagens está na Tabela 2.

Tabela 2 – Correspondência entre as construções do Domus e do C

Construção em Domus	Código gerado em C
sensor t: analogico(porta 1).	int t;
atuador c: digital(porta 2).	int c;
x <- expressao.	x = expressao;
ler(x).	scanf("%d", &x);
escrever(expressao).	printf("%d\n", expressao);
se c entao B fim_se	if (c) { B }
se c entao B1 senao B2 fim_se	if (c) { B1 } else { B2 }
enquanto (c) B fim_enquanto	while (c) { B }
+ - * /	+ - * /
< <= > >= == !=	< <= > >= == !=

Fonte: Autoria

A árvore sintática não guarda os parênteses escritos no programa fonte, porque a precedência dos operadores já está representada pela sua forma. Para que o código em C avalie a expressão na mesma ordem, toda subexpressão é emitida entre parênteses. A expressão w <- (6+8)*(5-7) resulta em w = ((6 + 8) * (5 - 7));.

Nenhum código é gerado se o programa fonte contiver erros: a compilação é interrompida ao final da análise sintática ou da análise semântica, conforme o caso.

5. COMO COMPILAR E EXECUTAR O CÓDIGO

5.1. Requisitos:

Um compilador C (gcc, MinGW ou equivalente), o gerador Flex e o gerador Bison. Em ambiente Linux (Debian/Ubuntu) os três são instalados com sudo apt install build-essential flex bison. Em ambiente Windows foi utilizado o MSYS2, com o comando pacman -S mingw-w64-ucrt-x86_64-gcc flex bison, acrescentando os diretórios C:\msys64\ucrt64\bin e C:\msys64\usr\bin à variável PATH. Nenhuma biblioteca externa é necessária.

5.2. Compilação

No diretório fonte execute:

bison -d domus.y

flex domus.l

gcc -o domusc main.c util.c symtab.c analyze.c code.c cgen.c lex.yy.c domus.tab.c

O bison é executado primeiro porque gera o arquivo domus.tab.h, que é incluído pelo global.h e, portanto, precisa existir antes da geração do analisador léxico.

5.3. Execução

O nome do programa em Domus é informado na linha de comando:

.\domusc prog_teste2.dms

A execução produz o arquivo listing.txt, com os tokens reconhecidos, a árvore sintática e a tabela de símbolos, e o arquivo prog_teste2.c, com o código gerado. Para compilar e executar o programa gerado:

gcc -o prog_teste2 prog_teste2.c

.\prog_teste2

6. RESULTADO DA EXECUÇÃO DO TRANSPILADOR

O transpilador foi exercitado com o programa de teste reproduzido na Figura 1, que corresponde ao exemplo fornecido no enunciado. O programa declara dois sensores e dois atuadores e utiliza os comandos de leitura, de escrita, de atribuição e de condição, nas duas formas, além de expressões com parênteses e comentários de uma e de várias linhas.

Figura 1 – Programa de teste prog_teste2.dms

```
1   /* Programa de teste */
2   sensor temperatura: analogico(porta 1).
3   sensor claridade:digital(porta 2).
4   /* multiplas
5   linhas
6   decomentario
7   */
8   atuador cortina: digital (porta 3).
9   atuador ventilador:analogico(porta 4).
10
11  ler(x).
12  w <- (6+8)*(5-7).
13  se (((temperatura < 30))) entao
14  cortina <- 1.
15  ventilador <- 80.
16  senao
17  cortina <- 0.
18  ventilador <- 0.
19  fim_se .
20  se claridade>20 entao cortina <- 0.fim_se.
21  asd <- 8 / x.
22  asd <- asd + 2 .
23  escrever(asd*2) .
```

Fonte: Teams da Equipe

A análise semântica produz a tabela de símbolos da Figura 2, exibida na tela e gravada na listagem. Cada entrada traz o nome do identificador, a categoria, o tipo, a porta, a posição de memória, a linha da declaração ou da primeira ocorrência e as

linhas de uso. Os quatro dispositivos ocupam as portas de 1 a 4, sem repetição, e as três variáveis foram criadas nas linhas em que receberam valor pela primeira vez.

Figura 2 – Tabela de símbolos gerada para o programa de teste

Nome	Categoria	Tipo	Porta	Local	Decl.	Linhas de uso
temperatura	sensor	analogico	1	0	2	13
claridade	sensor	digital	2	1	3	20
cortina	atuador	digital	3	2	8	14 17 20
ventilador	atuador	analogico	4	3	9	15 18
x	variavel	inteiro	-	4	11	21
w	variavel	inteiro	-	5	12	
asd	variavel	inteiro	-	6	21	22 23

Fonte: Autoria

O código em linguagem C gerado está na Figura 3. O programa obedece à estrutura exigida: a inclusão do cabeçalho stdio.h, os sensores, os atuadores e as variáveis declarados como inteiros globais e a função main com os comandos traduzidos.

Figura 3 – Código em linguagem C gerado pelo transpilador

```c
/*****************************************************************/
/* Codigo em linguagem C gerado pelo compilador DOMUS            */
/* Arquivo: prog_teste2.c                                        */
/*****************************************************************/

#include <stdio.h>

/* Sensores, atuadores e variaveis do programa */
int temperatura;      /* sensor analogico - porta 1 */
int claridade;        /* sensor digital - porta 2 */
int cortina;          /* atuador digital - porta 3 */
int ventilador;       /* atuador analogico - porta 4 */
int x;                /* variavel inteira */
int w;                /* variavel inteira */
int asd;              /* variavel inteira */

int main(void)
{
    scanf("%d", &x);
    w = ((6 + 8) * (5 - 7));
    if (temperatura < 30) {
        cortina = 1;
        ventilador = 80;
    } else {
        cortina = 0;
        ventilador = 0;
    }
    if (claridade > 20) {
        cortina = 0;
    }
    asd = (8 / x);
    asd = (asd + 2);
    printf("%d\n", (asd * 2));

    return 0;
}

```

Fonte: Autoria

O código gerado foi compilado com gcc -Wall -Wextra -o prog_teste2 prog_teste2.c sem nenhum aviso, atendendo à exigência do documento de regras. Executado com o valor 7, imprime 6, que é o resultado esperado.

O transpilador foi exercitado também com programas que contêm erros, para verificar o atendimento ao requisito de informar o tipo e a linha do erro. Os três casos foram reunidos no código abaixo, e a Figura 4 mostra o retorno obtido.

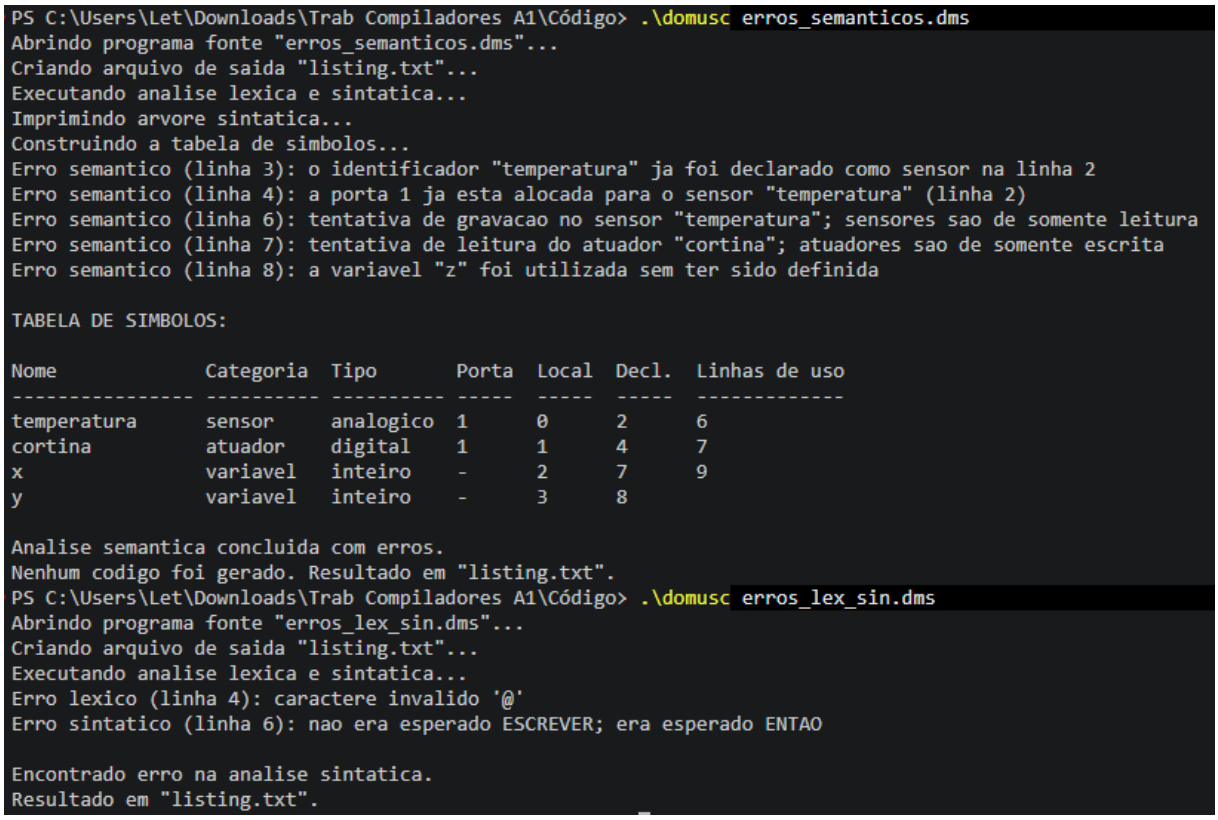
```
/* erros_semanticos.dms */
/* Teste de erros semanticos */
sensor temperatura: analogico(porta 1).
sensor temperatura: digital(porta 2).
atuador cortina: digital(porta 1).

temperatura <- 10.
x <- cortina + 1.
y <- z + 1.
escrever(x).

/* erros_lex_sin.dms */
/* Teste de erro lexico e de erro sintatico */
sensor temperatura: analogico(porta 1).

x <- temperatura @ 2.
se temperatura < 30
escrever(x).
fim_se.
```

Figura 4 – Mensagens de erro produzidas pelo transpilador



Fonte: Autoria

Os erros léxicos e sintáticos interrompem a compilação antes da análise semântica. Os erros semânticos não interrompem a varredura: todos são reportados e a tabela de símbolos é exibida, mas nenhum código é gerado.

REFERÊNCIAS

Aho, A. V., Lam, M. S., Sethi, R., Ullman, J. D. Compiladores: princípios, técnicas e ferramentas. 2ª ed. São Paulo: Pearson Addison-Wesley, 2008. Disponível na Biblioteca Virtual Pearson.

DONNELLY, C.; STALLMAN, R. Bison: the Yacc-compatible parser generator. Boston: Free Software Foundation, 2021. Disponível em: https://www.gnu.org/software/bison/manual/. Acesso em: 21 out. 2026.

Louden, K. C. Compiladores: princípios e práticas. São Paulo: Centage Learning, 2004. Disponível na Minha Biblioteca.

PAXSON, V.; ESTES, W.; MILLAWAY, J. Lexical Analysis with Flex: Flex 2.6.4 manual. 2017. Disponível em: https://westes.github.io/flex/manual/. Acesso em: 21 out. 2026.